

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший разработчик в X5 Tech

_____ В. Е. Козловский

« ____ » _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
"Программная инженерия", старший
преподаватель департамента
программной инженерии

_____ Н. А. Павлочев

« ____ » _____ 2026 г.

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.09 51 01-1-ЛУ

Исполнители:

Студент группы БПИ236

_____ / С. Е. Растворов /

« ____ » _____ 2026 г.

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.05.09 51 01-1-ЛУ

**РАЗРАБОТКА ИНТЕЛЛЕКТУАЛЬНОГО АССИСТЕНТА ДЛЯ
ОБРАЗОВАТЕЛЬНЫХ УЧРЕЖДЕНИЙ**

Программа и методика испытаний

RU.17701729.05.09 51 01-1

Листов 17

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Программа и методика испытаний определяет объект, цель, средства, порядок и методы испытаний программного продукта «Разработка интеллектуального ассистента для образовательных учреждений».

Документ предназначен для проверки соответствия программы требованиям технического задания и фиксации критериев приемки результата разработки.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения программы	4
2. ЦЕЛЬ ИСПЫТАНИЙ	5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	6
3.1. Требования к функциональным характеристикам	6
3.1.1. Состав выполняемых функций	6
3.1.2. Требования к входным данным	6
3.1.3. Требования к выходным данным	7
3.2. Требования к интерфейсу	7
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	8
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	9
5.1. Технические средства	9
5.2. Программные средства	9
5.3. Порядок проведения испытаний	9
5.4. Требования к персоналу	10
6. МЕТОДЫ ИСПЫТАНИЙ	11
6.1. Проверка требований к технической документации	11
6.2. Проверка требований к интерфейсу и функциональным характеристикам	11
6.2.1. Проверка допустимости вопроса	11
6.2.2. Проверка разграничения доступа	11
6.2.3. Проверка структуры ответа RAG	12
6.2.4. Проверка качества обычного RAG-ответа	12
6.2.5. Проверка резервного ответа при недоступности генеративной модели	13
6.2.6. Проверка пустого или недоступного векторного индекса	13
6.2.7. Проверка построения и обновления векторного индекса	13
6.2.8. Проверка поддерживаемых форматов документов	14
6.2.9. Проверка памяти веб-диалога и изоляции пользователей	14
6.2.10. Проверка обязательного контекста Telegram-диалога	14
6.2.11. Проверка метрик Prometheus	15
6.2.12. Проверка признака работоспособности процесса	15
6.3. Проверка автоматизированными средствами	15
СПИСОК ИСПОЛЪЗУЕМОЙ ЛИТЕРАТУРЫ	16

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «Разработка интеллектуального ассистента для образовательных учреждений».

Наименование программы на английском языке: «Development of an Intelligent Assistant for Educational Institutions».

1.2. Краткая характеристика области применения программы

Программа предназначена для интеллектуальной поддержки студентов и преподавателей НИУ ВШЭ и других вузов — ответы на вопросы об учебных планах, регламентах, сроках и процедурах на основе внутренних документов. Система базируется на RAG-архитектуре и использует локальную векторную базу (ChromaDB), языковую модель (Ollama/Mistral), эмбединги (ruBERT-tiny2) и интерфейсы Telegram/Web. Разработка выполнена командой трёх студентов НИУ ВШЭ в рамках курсового проекта по направлению 09.03.04 «Программная инженерия».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка соответствия программы «Разработка интеллектуального ассистента для образовательных учреждений» требованиям технического задания в части RAG-ядра, серверных интерфейсов, подготовки базы знаний, разграничения доступа и изоляции диалогов пользователей.

Испытания направлены на проверку следующих функций:

1. приём и проверка вопроса до вызова RAG-ядра;
2. векторный поиск релевантных фрагментов в ChromaDB, формирование ответа с помощью Ollama и возврат сведений об источниках;
3. формирование резервного ответа из фрагментов, уже найденных векторным поиском, если вызов языковой модели завершился ошибкой или превысил ограничение времени;
4. загрузка, нормализация и разбиение на фрагменты документов PDF, HTML, HTM, TXT и DOCX;
5. подготовка векторного индекса отдельной командой и автоматически при запуске сервера с пустым хранилищем;
6. возврат состояния процесса и технических метрик в формате Prometheus;
7. разграничение доступа по ключу API или веб-сессии, а также проверка роли аутентифицированного веб-пользователя при административных операциях;
8. поддержка ограниченного контекста диалогов веб- и Telegram-пользователей с изоляцией их историй и без доверия к произвольному идентификатору сессии из тела HTTP-запроса.

Проверка обязательных функций не зависит от их готовности на момент составления настоящего документа. Если обязательное испытание не имеет автоматизированного или ручного доказательства положительного результата, оно фиксируется как непройденное, а функция не исключается из приёмочного контроля.

Испытание считается пройденным, если фактический результат совпадает с ожидаемым результатом, указанным в разделе «Методы испытаний». Приёмка программы считается успешной, если все обязательные сценарии пройдены, не менее 85 % вопросов контрольного набора получили корректные содержательные ответы, а среднее время ответа метода POST /ask при доступном Ollama не превысило 7 секунд. Указанные значения являются целевыми критериями и не считаются достигнутыми без протокола фактического прогона.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Состав выполняемых функций

В ходе испытаний проверяются следующие функции RAG-ядра и серверной части программы «Разработка интеллектуального ассистента для образовательных учреждений».

1. Метод `POST /ask` принимает вопрос после проверки ключа API в заголовке `X-API-Key`. Переданный в теле запроса `session_id` проверяется по типу и длине, но не используется как доверенный ключ памяти.
2. Метод `POST /web/ask` принимает вопрос при наличии корректного `X-API-Key` или действующей веб-сессии. Контекст диалога используется только при наличии веб-пользователя, определённого сервером по cookie `web_session`. Ключ API без такой сессии не создаёт пользовательский контекст.
3. Программа формирует векторное представление запроса моделью `cointegrated/rubert-tiny2` и извлекает ближайшие по векторной мере фрагменты из ChromaDB.
4. Программа передаёт вопрос, ограниченную историю вопросов и найденные фрагменты модели Ollama, после чего возвращает ответ и сведения об источниках.
5. Если векторный поиск завершён успешно, но Ollama недоступна или превышено ограничение времени, программа формирует резервный ответ из уже найденных фрагментов.
6. Модуль `document_ingestion.py` загружает PDF, HTML, HTM, TXT и DOCX, нормализует текст, делит его на перекрывающиеся фрагменты и сохраняет их с метаданными в ChromaDB. Индекс готовится командой модуля `index_documents.py` или автоматически при запуске с пустым хранилищем.
7. Модуль `conversation_memory.py` хранит только предыдущие вопросы в ограниченном окне и сроке жизни. Для веб-пользователя сервер формирует ключ `web:<user_id>`. В целевой версии для Telegram-пользователя сервер должен формировать ключ `tg:<telegram_id>` из идентификатора, полученного через доверенный контур бота.
8. Модуль `metrics.py` формирует счётчики запросов, резервных ответов и ошибок, а также гистограммы длительности поиска, генерации и полной обработки.
9. Ключ API проверяет право вызова защищённого метода, но не назначает роль. Роли «пользователь» и «администратор» относятся к аутентифицированным веб-пользователям; только администратор может создавать приглашения.
10. Метод `GET /health` подтверждает живость HTTP-процесса. Он не подтверждает готовность Ollama, ChromaDB и баз данных.

3.1.2. Требования к входным данным

Входными данными серверной части являются:

1. тело запросов `POST /ask` и `POST /web/ask` в формате JSON с обязательным полем `question` и необязательным полем `session_id`; вопрос после удаления крайних пробельных символов должен быть непустой строкой длиной не более 500 символов;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. заголовок X-API-Key для доступа к POST /ask, GET /metrics и, как к одному из вариантов, к POST /web/ask;
3. cookie web_session, создаваемая после входа, первичного создания администратора или принятия приглашения;
4. данные HTML-форм для POST /web/login, POST /web/bootstrap, POST /web/invite/accept, POST /web/admin/invites и POST /web/logout;
5. документы корпуса форматов PDF, HTML, HTM, TXT и DOCX из каталога, заданного DOCUMENTS_DIR или аргументом --input-dir;
6. переменные окружения API_KEY, OLLAMA_HOST, LLM_MODEL, HF_EMBEDDING_MODEL, VECTOR_DB_DIR, DOCUMENTS_DIR, WEB_AUTH_DATABASE_URL, DATABASE_URL, а также параметры окна и срока жизни диалогов.

3.1.3. Требования к выходным данным

Выходными данными серверной части являются:

1. JSON-ответы POST /ask и POST /web/ask с верхнеуровневыми полями answer, sources и metadata;
2. элементы массива sources с полем content и объектом metadata, который может содержать название, источник, страницу и номер фрагмента;
3. объект metadata с полями model, embedding_model, num_sources, confidence, fallback_used, fallback_reason, retrieval_time_ms, generation_time_ms и total_time_ms;
4. ответы об ошибках в формате JSON с полем error и, для отдельных ошибок RAG-ядра, полем code;
5. текстовые метрики Prometheus для GET /metrics, включая rag_requests_total, rag_fallback_total, rag_errors_total, rag_retrieval_seconds, rag_generation_seconds и rag_total_seconds;
6. JSON-ответ GET /health с кодом HTTP 200 и телом {"status": "ok"}.

3.2. Требования к интерфейсу

Сервер реализован средствами FastAPI и предоставляет следующие HTTP-методы.

1. POST /ask — обработка вопроса при наличии X-API-Key без пользовательской памяти.
2. POST /web/ask — обработка вопроса при наличии X-API-Key или действующей веб-сессии; память применяется только к определённому по cookie веб-пользователю.
3. GET /health — проверка живости HTTP-процесса без аутентификации.
4. GET /metrics — возврат технических метрик Prometheus при наличии X-API-Key.
5. GET /, GET /web, POST /web/login, POST /web/bootstrap, POST /web/invite/accept, POST /web/admin/invites и POST /web/logout — методы веб-интерфейса, входа, первичной настройки, принятия приглашения, создания приглашения администратором и выхода.

При отсутствии или неверном ключе API защищённые им HTTP-методы возвращают код 401. Метод POST /web/ask также возвращает код 401, если нет ни корректного ключа API, ни действующей веб-сессии. Попытка неадминистратора создать приглашение возвращает код 403. Интерактивная документация API формируется FastAPI и доступна по адресу /docs.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытание должна быть представлена документация в следующем составе:

1. «Разработка интеллектуального ассистента для образовательных учреждений». Техническое задание (ГОСТ 19.201-78 [7]).
2. «Разработка интеллектуального ассистента для образовательных учреждений». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «Разработка интеллектуального ассистента для образовательных учреждений». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «Разработка интеллектуального ассистента для образовательных учреждений». Текст программы (ГОСТ 19.401-78 [9]).
5. «Разработка интеллектуального ассистента для образовательных учреждений». Руководство оператора (ГОСТ 19.505-79 [11]).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Испытания проводятся на рабочей станции, на которой могут быть запущены Python-окружение проекта, Docker Compose и Ollama с заданной языковой моделью. Объём оперативной памяти и вычислительные ресурсы должны соответствовать требованиям выбранной модели. Графический ускоритель может использоваться для сокращения времени генерации, но не является обязательным условием функциональных испытаний.

В состав стенда входят FastAPI-сервер, Ollama, ChromaDB с сохраняемым на диске векторным индексом, веб-клиент, Telegram-бот и PostgreSQL 16. По умолчанию веб-пользователи, веб-сессии и ограниченная память вопросов хранятся в SQLite по адресу WEB_AUTH_DATABASE_URL. PostgreSQL по адресу DATABASE_URL хранит учётные записи Telegram-пользователей и полный журнал обращений бота. Для ручной проверки API используется HTTP-клиент, например curl или Postman.

5.2. Программные средства

Зависимости проекта и инструменты разработки устанавливаются и запускаются с помощью uv. Команда `uv run -m pytest` запускает модуль `pytest` в окружении проекта. Параметр `-q` сокращает служебный вывод, не изменяя состав выполняемых тестов.

```
uv sync --all-packages --group dev
PYTHONPATH=. uv run -m pytest tests/test_ask.py tests/test_question_validation.py \
  tests/test_rag.py tests/test_document_ingestion.py tests/test_health.py \
  tests/test_server_config.py -q
./uv-linters.sh
docker compose config
```

Для ручной подготовки векторного индекса используется следующая команда:

```
PYTHONPATH=. uv run python -m src.server.app.index_documents \
  --input-dir "$(pwd)/data_and_documents" \
  --persist-dir "$(pwd)/src/server/chrome_langchain_db" \
  --rebuild
```

Активация окружения командой `source .venv/bin/activate` не требуется, поскольку `uv run` самостоятельно выбирает окружение проекта.

5.3. Порядок проведения испытаний

Испытания выполняются в следующем порядке.

1. Проверить наличие `uv`, `Docker` и `Docker Compose`, а для полного прогона RAG — запущенного Ollama с моделью, указанной в `LLM_MODEL`.
2. Установить зависимости командой `uv sync --all-packages --group dev`.
3. Создать `.env` на основе `.env.example`. Задать как минимум `API_KEY`, `OLLAMA_HOST`, `VECTOR_DB_DIR`, `DOCUMENTS_DIR`, `WEB_AUTH_DATABASE_URL`, `DATABASE_URL` и параметры PostgreSQL. Для веб-стенда также задать `WEB_BOOTSTRAP_ADMIN_TOKEN`, для Telegram-бота — `BOT_TOKEN`.
4. Выполнить `./uv-linters.sh`, затем запустить указанные автоматизированные тесты. Каждая команда должна завершиться с кодом 0.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. Проверить конфигурацию командой `docker compose config` и запустить стенд командой `docker compose --profile dev up -d`.
6. Подготовить индекс отдельной командой с `--rebuild` либо проверить автоматическую индексацию пустого хранилища при `PREPARE_RAG_ON_STARTUP=1` и `AUTO_INDEX_ON_STARTUP=1`.
7. Проверить `GET /health`, затем методы `POST /ask`, `POST /web/ask` и `GET /metrics` с соответствующими вариантами аутентификации.
8. Выполнить сценарии изоляции веб-диалогов и ограничения окна памяти. Обязательный сценарий Telegram-контекста фиксируется как непройденный, если безопасный серверный ключ `tg:<telegram_id>` не реализован.
9. После успешного векторного поиска сделать Ollama недоступной и повторить запрос. Проверить резервный ответ из найденных фрагментов и признак `fallback_used=true`.
10. Выполнить контрольный набор вопросов, зафиксировать ответы, источники и `total_time_ms`, после чего рассчитать долю корректных ответов и среднее время.
11. Внести фактические результаты, коды завершения команд и ссылки на доказательства в протокол испытаний. При отсутствии доказательства по обязательному сценарию его результат не считается положительным.

5.4. Требования к персоналу

Испытания должен проводить разработчик или инженер по тестированию, умеющий запускать Python-проекты с помощью `uv`, работать с Docker Compose, выполнять HTTP-запросы и анализировать журналы сервисов. Экспертная оценка корректности ответов выполняется путём сопоставления ответа с содержанием указанных в нём источников. Для проверки порога 85 % должны быть заранее подготовлены контрольные вопросы и эталонные сведения из документов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Проверка требований к технической документации

Проверку технической документации выполняют сопоставлением технического задания, пояснительной записки, руководства оператора и настоящей программы и методики испытаний с исходным кодом и автоматизированными тестами. При сверке используют модули `rag.py`, `vector.py`, `document_ingestion.py`, `index_documents.py`, `conversation_memory.py`, `metrics.py`, `auth_crud.py`, `auth_database.py`, `auth_models.py`, `config.py` и маршруты приложения в `main.py`.

Результат проверки считают положительным, если одинаковые функции, маршруты, входные и выходные данные, способы аутентификации и ограничения системы описаны во всех документах единообразно. Обязательные функции, которые ещё не реализованы или не подтверждены испытанием, сохраняют в документации как целевые требования и отмечают в протоколе как непройденные, а не исключают из состава системы.

6.2. Проверка требований к интерфейсу и функциональным характеристикам

Каждый сценарий содержит исходные данные, действия, ожидаемый результат, критерий успешности и доказательство. Ссылки на автоматизированные тесты указывают существующее средство проверки. Если сценарий требует ручного протокола или ещё не реализованного интеграционного теста, положительный результат не допускается фиксировать только на основании описания.

6.2.1. Проверка допустимости вопроса

Исходные данные: значения поля `question`: `null`, число, пустая строка, строка из пробельных символов, строка длиной 501 символ и строка длиной 500 символов.

Действия: последовательно передать значения функции нормализации и маршруту `POST /ask`; при проверке недопустимых значений подменить обращение к RAG функцией, аварийно завершающей тест при вызове.

Ожидаемый результат: недопустимые значения отклоняются с HTTP 400 до обращения к RAG; строка длиной 500 символов принимается и передаётся в обработку после удаления пробельных символов по краям.

Критерий успешности: все граничные и отрицательные варианты дают предусмотренный результат, а RAG не вызывается для недопустимого вопроса.

Доказательство: `tests/test_question_validation.py`, тесты проверки вопроса и границы длины в `tests/test_ask.py`.

6.2.2. Проверка разграничения доступа

Исходные данные: правильное, отсутствующее и неправильное значение заголовка `X-API-Key`; действующая и отсутствующая веб-сессия; веб-пользователь без роли администратора.

Действия: выполнить запросы к `POST /ask` и `GET /metrics` с каждым вариантом ключа; выполнить `POST /web/ask` с ключом API, с веб-сессией и без учётных данных; от имени обычного веб-пользователя попытаться создать приглашение через `POST /web/admin/invites`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Ожидаемый результат: правильный ключ предоставляет доступ к защищённым маршрутам; отсутствующий или неправильный ключ отклоняется с HTTP 401; POST /web/ask принимает правильный ключ API либо действующую веб-сессию; запрос без обоих способов доступа отклоняется; создание приглашения обычным пользователем отклоняется с HTTP 403.

Критерий успешности: коды состояния совпадают с ожидаемыми, а административная операция недоступна пользователю без роли администратора. Ключ API не рассматривается как носитель пользовательской роли.

Доказательство: тесты аутентификации маршрутов, test_web_ask_requires_authentication, test_web_ask_accepts_api_key и test_non_admin_cannot_create_invites в tests/test_ask.py; тесты метрик в tests/test_health.py.

6.2.3. Проверка структуры ответа RAG

Исходные данные: допустимый вопрос и управляемая тестовая реализация функции RAG, возвращающая ответ, сведения об источнике и метаданные обработки.

Действия: выполнить POST /ask с правильным ключом API и проверить JSON-ответ.

Ожидаемый результат: верхний уровень ответа содержит поля answer, sources, metadata; каждый элемент sources содержит content и metadata; объект metadata ответа содержит сведения о модели, числе источников, достоверности, резервном режиме и длительности этапов обработки.

Критерий успешности: имена полей, их вложенность и типы соответствуют модели ответа API.

Доказательство: test_ask_returns_compatible_contract в tests/test_ask.py.

6.2.4. Проверка качества обычного RAG-ответа

Исходные данные: проиндексированный корпус документов образовательной организации, работающие ChromaDB и Ollama, контрольный набор не менее чем из 20 вопросов с эталонными ответами и указанием документов-источников.

Действия: направить каждый вопрос через POST /ask; для каждого ответа зафиксировать текст, источники, значение confidence и total_time_ms; вручную сопоставить ответ и источники с эталоном.

Ожидаемый результат: ответ основан на содержимом найденных документов, список источников не содержит повторов одного фрагмента, значение достоверности находится в диапазоне от 0 до 1, а длительность обработки зафиксирована.

Критерий успешности: не менее 85 % вопросов получают корректный ответ, а среднее время обработки не превышает 7 секунд на указанном испытательном стенде.

Доказательство: заполненный ручной протокол по контрольному набору; test_ask_returns_compatible_contract в tests/test_ask.py и test_deduplicate_sources_uses_source_page_and_chunk в tests/test_rag.py. До получения заполненного протокола целевые показатели 85 % и 7 секунд не считают подтверждёнными.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6.2.5. Проверка резервного ответа при недоступности генеративной модели

Исходные данные: работоспособный векторный поиск и искусственно вызванная ошибка либо превышение времени ожидания Ollama после получения релевантных фрагментов.

Действия: выполнить допустимый запрос и зафиксировать код состояния, текст ответа, список источников, `fallback_used` и `fallback_reason`.

Ожидаемый результат: сервер возвращает HTTP 200 и формирует резервный ответ из уже найденных векторным поиском фрагментов; `fallback_used` имеет значение `true`, а `fallback_reason` содержит причину перехода в резервный режим.

Критерий успешности: отказ генеративной модели не приводит к HTTP 5xx, ответ содержит найденные сведения и признаки резервного режима.

Доказательство: `test_ask_question_returns_fallback_when_llm_fails` в `tests/test_rag.py`; ручной протокол проверки превышения времени ожидания Ollama.

6.2.6. Проверка пустого или недоступного векторного индекса

Исходные данные: пустой индекс ChromaDB и отдельно недоступное хранилище индекса.

Действия: выполнить допустимый запрос к POST /ask в каждом состоянии хранилища.

Ожидаемый результат: сервер возвращает HTTP 503 с машинно-читаемым кодом `vector_index_empty` для пустого индекса либо `vector_store_unavailable` для недоступного хранилища. Резервный ответ без найденных фрагментов не формируется.

Критерий успешности: состояние инфраструктуры однозначно отражено кодом HTTP и полем `code`, а ответ не выдаётся за результат поиска по документам.

Доказательство: `test_ask_returns_503_for_empty_index` в `tests/test_ask.py`; ручной протокол отключения хранилища для варианта `vector_store_unavailable`.

6.2.7. Проверка построения и обновления векторного индекса

Исходные данные: каталог поддерживаемых документов, пустой каталог индекса и ранее построенный индекс.

Действия: выполнить `uv run python -m src.server.app.index_documents` с параметрами `--input-dir`, `--persist-dir` и `--rebuild`; повторить индексирование без изменения документов; отдельно запустить сервер с разрешённой автоматической подготовкой пустого индекса.

Ожидаемый результат: после первой обработки индекс содержит фрагменты документов; повторный запуск не создаёт дубликаты; режим `--rebuild` удаляет устаревшее содержимое; сервер при запуске подготавливает только пустой индекс, если соответствующая настройка включена.

Критерий успешности: команда завершается с кодом 0, количество и состав записей соответствуют исходным документам, повторная обработка идемпотентна.

Доказательство: тесты индексирования, повторного запуска, перестроения и удаления устаревших документов в `tests/test_document_ingestion.py`; тест автоматической подготовки пустого индекса в `tests/test_health.py`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6.2.8. Проверка поддерживаемых форматов документов

Исходные данные: файлы PDF, HTML, HTM, TXT и DOCX с известным текстом, а также настроенные размер фрагмента и перекрытие.

Действия: обработать каждый файл модулем загрузки документов; проверить извлечённый текст, метаданные, длину и взаимное перекрытие соседних фрагментов.

Ожидаемый результат: текст извлекается из каждого поддерживаемого формата; HTML-разметка не попадает в содержимое; кодировка TXT определяется корректно; фрагменты не превышают настроенный размер и сохраняют заданное перекрытие; метаданные позволяют определить исходный документ.

Критерий успешности: каждый поддерживаемый формат даёт непустые фрагменты с ожидаемым текстом и метаданными. Распознавание текста на изображениях в данный сценарий не входит.

Доказательство: автоматизированные проверки TXT, HTML, DOCX, разбиения и метаданных в tests/test_document_ingestion.py; ручной протокол для PDF и HTM.

6.2.9. Проверка памяти веб-диалога и изоляции пользователей

Исходные данные: две действующие веб-сессии пользователей А и В; вопросы А1, А2, В1 и В2; последовательность дополнительных вопросов одного пользователя, превышающая пять сообщений.

Действия: выполнить запросы POST /web/ask в порядке А1, В1, А2, В2 и зафиксировать историю, переданную в RAG; затем проверить ограничение окна памяти; отдельно выполнить два запроса с ключом API и произвольным клиентским session_id без веб-сессии.

Ожидаемый результат: истории запросов А1, В1, А2 и В2 равны соответственно [], [], [А1], [В1]; в памяти остаются последние пять вопросов пользователя; произвольный session_id запроса с одним ключом API не создаёт доверенную пользовательскую память.

Критерий успешности: история одного веб-пользователя не передаётся другому, ограничение окна соблюдается, а клиентский идентификатор не используется как ключ доверенного контекста.

Доказательство: test_web_ask_uses_web_user_memory_key, test_web_ask_memory_isolated_between_users, test_web_ask_session_memory_keeps_last_five_messages, test_ask_does_not_use_caller_controlled_session_memory и test_web_ask_with_api_key_does_not_use_caller_controlled_session_memory в tests/test_ask.py.

6.2.10. Проверка обязательного контекста Telegram-диалога

Исходные данные: два пользователя Telegram с различными числовыми идентификаторами; вопросы А1, А2, В1 и В2; доверенный интерфейс между ботом и сервером, в котором сервер получает проверенную идентичность пользователя.

Действия: выполнить обращения через Telegram в порядке А1, В1, А2, В2; перехватить истории, переданные RAG; попытаться подменить идентификатор пользователя через общедоступное поле запроса к POST /ask.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Ожидаемый результат: сервер самостоятельно формирует ключи вида `tg:<telegram_id>`; истории четырёх запросов равны `[], [], [A1], [B1]`; подмена клиентом идентификатора игнорируется либо отклоняется.

Критерий успешности: контекст сохраняется отдельно для каждого пользователя Telegram и не может быть присвоен другому пользователю через управляемое клиентом поле.

Доказательство: положительный интеграционный тест доверенного маршрута Telegram и отрицательный тест подмены идентификатора. На текущей ревизии такие серверный механизм и тесты отсутствуют, поэтому испытание не пройдено. Тест передачи `session_id` клиентом Telegram подтверждает только отправку значения и не является доказательством безопасной серверной памяти.

6.2.11. Проверка метрик Prometheus

Исходные данные: не менее пяти обработанных запросов и правильный ключ API.

Действия: выполнить `GET /metrics` с правильным ключом и без ключа; проверить формат и наличие метрик `rag_requests_total`, `rag_fallback_total`, `rag_errors_total`, `rag_retrieval_seconds`, `rag_generation_seconds`, `rag_total_seconds`.

Ожидаемый результат: авторизованный запрос возвращает HTTP 200 и текст в формате Prometheus; запрос без ключа возвращает HTTP 401; счётчики и гистограммы доступны под установленными именами.

Критерий успешности: маршрут защищён, ответ разбирается как метрики Prometheus и содержит перечисленные семейства метрик.

Доказательство: тесты маршрута метрик в `tests/test_health.py`.

6.2.12. Проверка признака работоспособности процесса

Исходные данные: запущенный FastAPI-сервер.

Действия: выполнить неаутентифицированный `GET /health`.

Ожидаемый результат: сервер возвращает HTTP 200 и JSON `{"status": "ok"}`.

Критерий успешности: ответ подтверждает, что процесс приложения принимает HTTP-запросы. Этот маршрут является проверкой работоспособности процесса и не подтверждает готовность ChromaDB, Ollama или баз данных.

Доказательство: `test_health_ok` в `tests/test_health.py`.

6.3. Проверка автоматизированными средствами

Для подтверждения существующих автоматизированных сценариев выполняют команды, приведённые в разделе «Средства и порядок испытаний». Результат считают положительным, если `pytest`, статические проверки и проверка конфигурации Docker Compose завершаются с кодом 0. Ручные сценарии и непройденное испытание Telegram фиксируют в отдельном протоколе; успешный автоматизированный прогон не заменяет эти результаты.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.09 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]